

Tamper-evident Credential Registry

*Change one byte of a degree certificate and anyone
can tell — in seconds, for free.*

<Your Name> • Roll <Roll No> • M.Tech CSE, Sem 3

Blockchain Technologies (24MTSCSEDE53)



0x493f...cb70

VALID ✓

Forged degrees: cheap to make, slow to check

Who suffers today



Employers

cannot cheaply tell a genuine PDF from an edited one



Honest graduates

wait days while an offer is on hold



Universities

answer the same verification e-mails again and again

One concrete failure

Master of Technology (CSE)

Grade: ~~Second Class~~ **First Class**

Edited in any PDF editor. Logo, signature and layout are untouched, so it looks perfect.



To check it, the employer e-mails the registrar or pays an agency, and waits days. Most never check.

Why blockchain? Three questions, answered



Who are the mutually distrusting writers?

Several universities write to one shared registry. None may edit, backdate or delete another's records, and none should run it for the others.



Which intermediary is removed?

The verification agency and the "e-mail the registrar and wait" loop. Anyone re-hashes the PDF and reads the chain for free, without asking anyone.



Would a database + signatures do?

Partly. A signature proves who issued it. But a university-run revocation list can be silently edited, backdated or taken offline. The chain gives one append-only status that no single university controls.

Architecture: hashes on-chain, people off-chain



Registrar



Verifier



Student

Python client (off-chain)

cli.py

domain · cohort

hashing · merkle

chain.py — the only web3 import

JSON-RPC · local node

CredentialRegistry.sol (on-chain)

AccessControl · REGISTRAR_ROLE · Sha256Merkle · events

ON-CHAIN

docHash = SHA-256(PDF)

subjectHash (salted id)

issuer · times · status

cohort Merkle root

OFF-CHAIN

PDF, name, student id

salt + Merkle proof (student keeps)

revocation reason text

Module map and contract interface

contracts/

- | CredentialRegistry.sol
- | Sha256Merkle.sol

client/

- | config.py constants
- | chain.py web3 only
- | domain · cohort
- | hashing · merkle
- | cli · models

scripts/ deploy · demo · tamper

tests/ 10 sol + 5 py

Data model

```
enum Status { NONE, VALID, REVOKED }  
Credential { subjectHash, issuer,  
            issuedAt, revokedAt, status }
```

Contract interface

issue / revoke (docHash, ...) registrar only

verify (docHash) free view, never reverts

issueBatch (root, size) one tx per cohort

revokeBatchMember (... , proof) one student

verifyBatchMember (... , proof) free view

Live demo: make node · make demo

1

Issue

ISSUED

2

Verify

VALID

3

Flip 1 byte

forged

4

Verify forged

FAIL

5

Revoke

tx sent

6

Verify again

REVOKED

7

Outsider issues

REJECTED

VALID: issued 2026-09-28 by 0x7099...79C8

FAIL: not registered - the file was altered or never issued

REVOKED: issued 07:44:12 UTC ... revoked 07:44:14 UTC

REJECTED by contract: AccessControlUnauthorizedAccount

Stretch: make demo-cohort — 3 students, one Merkle root; revoke one, others stay VALID.

Security, tests, limits and AI use

Threat	Control	Residual risk
Stolen registrar key	Role check; admin removes role; issuer recorded	Fakes stay VALID until revoked; admin should be multisig
Brute-force id hash from roll list	32-byte random salt per student; no PII on-chain	Leaked salt links the student; hashes are permanent
Look-alike fake registry	Verifiers use the published contract address	Trust moves to how that address is shared

15

tests, 9 negative — one command: make test

94k → 49k

gas: one credential vs a whole cohort. Limit: a re-saved PDF fails; proves issuer, not merit

AI

Claude Code for code, tests, docs, slides — declared in AI_USAGE.md, 4 AI mistakes caught